Data source excerpt (continued) *data_source.py*

```
209
210
211 def _clean_records(records):
212     filtered = [row for row in records if _is_useful_record(row)]
213     by_callsign = {}
214
215     for row in filtered:
216         callsign = str(row.get("callsign", "")).strip().upper()
217         icao = str(row.get("icaoAddress", "")).strip().upper()
218         key = icao if icao and icao != "UNKNOWN" else callsign
219         if not key:
220             continue
221
222         current = by_callsign.get(key)
223         if current is None:
224             by_callsign[key] = row
225             continue
226
227         def score(rec):
228             lat = float(rec.get("latDD", 0) or 0)
229             lon = float(rec.get("lonDD", 0) or 0)
230             altitude = float(rec.get("altitudeMM", 0) or 0)
231             speed = float(rec.get("horVelocityCMS", 0) or 0)
232             heading = float(rec.get("headingDE2", 0) or 0)
233             squawk = int(rec.get("squawk", 0) or 0)
234             frame_len = int(rec.get("frameLength", 0) or 0)
235             coord_score = 100 if (lat != 0 and lon != 0) else 0
236             altitude_score = 10 if altitude > 0 else 0
237             speed_score = 5 if speed > 0 else 0
238             heading_score = 5 if heading > 0 else 0
239             squawk_score = 2 if squawk > 0 else 0
240             return coord_score + altitude_score + speed_score + heading_score + squawk_score + frame_len
241
242         if score(row) >= score(current):
243             by_callsign[key] = row
244
245     return list(by_callsign.values())
246
247
248 def _load_replay_entries():
249     global _replay_entries, _replay_index, _replay_file_mtime, _replay_file_path
250
251     path = Path(_get_simulation_log_file())
252     if not path.exists():
253         _replay_entries = []
254         _replay_index = 0
255         return
256
257     mtime = path.stat().st_mtime
258     path_str = str(path.resolve())
259     should_reparse = (
260         _replay_file_mtime is None
261         or _replay_file_mtime != mtime
262         or _replay_file_path != path_str
263         or not _replay_entries
264     )
265     if not should_reparse:
266         return
267
268     _replay_file_mtime = mtime
269     _replay_file_path = path_str
270     _replay_entries = _decoder.parse_log_entries(str(path), max_entries=None)
271     _replay_index = 0
272     _decoder.reset_state()
273
274     # Warm up the state so the very first API response has richer fields
275     # (heading/speed/squawk) for flights that start with sparse packets.
276     # Reset afterwards so real replay (starting again at index 0) builds each
277     # aircraft's timestamp forward from its first appearance ? otherwise the
278     # warmup leaves "timestamp" at each aircraft's *last* appearance, and the
279     # very first real batch immediately overwrites it with an earlier
280     # timestamp, storing an out-of-order trajectory point that later makes
281     # the predictor extrapolate from the wrong segment of the route.
282     warmup = len(_replay_entries)
283     for i in range(warmup):
284         raw_time, source_ip, payload, callsign_hint = _replay_entries[i]
285         _decoder.decode_payload(
286             payload,
287             source_ip=source_ip,
288             timestamp=raw_time,
289             callsign_hint=callsign_hint,
290         )
291     _decoder.reset_state()
292     _replay_index = 0
293
294
295 def _get_replayed_snapshot():
296     global _replay_index
297
298     _load_replay_entries()
299     if not _replay_entries:
300         return [], [], []
301
302     batch = max(1, SIMULATION_REPLAY_BATCH)
303     processed = 0
304     decoded_events = []
305
306     while processed < batch:
307         if _replay_index >= len(_replay_entries):
308             if not SIMULATION_REPLAY_LOOP:
309                 break
310             _replay_index = 0
311             _decoder.reset_state()
312
```